

# Core 4 — Chips & AI Hackathon Project Plan

## In-Memory Acceleration of Monte Carlo and ML-Based Financial Workloads

Nini Hsu • Rupsaa Goswami • Raya Chauhan • Ishika Pandurangam

Theme 4: In-Memory Computing and Circuits | Final demo video due Sep 19, 2026

## 1. Project Goal

We will build a software proof of concept that uses option pricing as a financial workload. Monte Carlo simulation will generate reference option prices and training data. A small neural-network surrogate will learn to approximate those prices. We will then model how the neural network's matrix-vector computations could map to an in-memory or near-memory computing architecture and compare accuracy and computational efficiency.

## 2. What We Are — and Are Not — Modeling

We are not predicting the future price of a particular company such as Apple or NVIDIA. The core dataset will contain synthetic, hypothetical European call-option scenarios. Each scenario is defined by standard option-pricing parameters. This keeps the project focused on computational workload acceleration rather than stock forecasting.

A real stock may be used later as a simple demonstration example, but it is not required for training.

## 3. End-to-End Pipeline

Synthetic Option Scenarios → Monte Carlo Pricing → Training Dataset  
→ Neural-Network Surrogate → IMC Mapping → Performance Comparison

## 4. Dataset Specification

Each row represents one hypothetical European call option. The five model inputs are S, K, T, sigma, and r. The Monte Carlo price is the target label that the neural network will learn.

| Column   | Meaning                         | Initial Range             | Role            |
|----------|---------------------------------|---------------------------|-----------------|
| S        | Current underlying price        | \$50–\$200                | NN input        |
| K        | Strike price                    | Chosen to cover moneyness | NN input        |
| T        | Time to expiration (years)      | 0.05–2.0                  | NN input        |
| sigma    | Annualized volatility           | 0.10–0.60                 | NN input        |
| r        | Risk-free rate                  | 0.00–0.08                 | NN input        |
| mc_price | Monte Carlo European call price | Calculated                | Training target |

Sampling note: instead of choosing S and K blindly, we should deliberately cover moneyness S/K, approximately 0.7–1.3, so the dataset contains out-of-the-money, at-the-money, and in-the-money options.

Useful metadata to store for benchmarking: num\_paths, mc\_runtime\_ms, and moneyness. These do not have to be NN inputs.

Initial development dataset: 500–1,000 rows. Final target: approximately 20,000–100,000 scenarios, adjusted based on runtime and project constraints.

## 5. Monte Carlo Reference Model

For each scenario, simulate terminal stock prices under the standard risk-neutral geometric Brownian motion model:

$$S_T = S \cdot \exp[(r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T} \cdot Z], \quad Z \sim N(0,1)$$

For a European call, the payoff for each simulated terminal price is:

$$\text{Payoff} = \max(S_T - K, 0)$$

The discounted average payoff is the Monte Carlo option-price estimate:

$$C_{MC} = \exp(-rT) \cdot \text{average}[\max(S_T - K, 0)]$$

First validation case:  $S=100$ ,  $K=100$ ,  $T=1$ ,  $\sigma=0.20$ ,  $r=0.05$ . With many paths, Monte Carlo should approach the Black–Scholes value of about \$10.45. This gives us a sanity check before generating a large dataset.

## 6. Team Responsibilities

### Nini Hsu — Monte Carlo & Financial Modeling

- Implement the European call Monte Carlo pricer.
- Validate Monte Carlo output against Black–Scholes for test cases.
- Define/generate synthetic parameter scenarios and produce the CSV dataset.
- Record Monte Carlo runtime and number of paths.
- Explain the financial workload and why repeated Monte Carlo pricing is computationally intensive.
- Handoff: provide Rupsaa with the real dataset in the agreed format.

### Rupsaa Goswami — Neural-Network Surrogate

- Build the NN pipeline immediately using dummy/test data in the agreed format.
- Load and preprocess  $S$ ,  $K$ ,  $T$ ,  $\sigma$ ,  $r$ ; split train/validation/test data.
- Start with a small architecture such as  $5 \rightarrow 64 \rightarrow 64 \rightarrow 1$ .
- Train on Nini's final Monte Carlo labels when available.
- Measure MAE, RMSE,  $R^2$ , inference latency, parameter count, and model size.
- Handoff: provide Raya with the final NN layer dimensions/weights/precision assumptions and Ishika with accuracy/runtime results.

### Raya Chauhan — IMC Architecture & Mapping

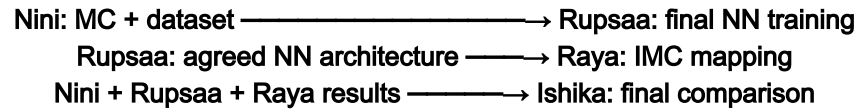
- Choose and document a defensible software-level IMC/near-memory modeling approach.
- Model the NN's dense layers as matrix-vector operations  $y = Wx + b$ .
- Calculate MAC counts, memory accesses, and estimated data movement.
- Estimate/model latency and energy under conventional vs IMC assumptions.
- Clearly label simulated/estimated results and document all assumptions.
- Handoff: provide Ishika with final IMC metrics and assumptions.

### Ishika Pandurangam — Benchmarking, Analysis & Visualization

- Create a common benchmarking/results format while other components are being built.
- Benchmark Monte Carlo and NN conventional-compute runtime/throughput.
- Collect outputs from all team members into comparable tables.
- Create accuracy, latency/speedup, data-movement, energy, and throughput plots as supported by the results.
- Prepare the central MC vs NN CPU vs NN+IMC comparison.
- Coordinate final figures for the demo/video.

## 7. Parallel Work and Dependencies

The project has dependencies, but development should happen in parallel. Rupsaa does not need to wait for the final Monte Carlo dataset to build the NN code; she can use a temporary CSV with the same columns. Raya can begin the IMC framework once the tentative NN architecture is frozen. Ishika can build benchmark and plotting functions before final measurements exist.



## 8. Final Schedule

| Date   | Nini                                      | Rupsaa                           | Raya                           | Ishika                        | Milestone               |
|--------|-------------------------------------------|----------------------------------|--------------------------------|-------------------------------|-------------------------|
| Sep 9  | MC model, validation case, dataset format | NN pipeline with dummy data      | Choose IMC modeling approach   | Define metrics/results format | Freeze scope/interfaces |
| Sep 10 | Implement reusable MC pricer; small CSV   | Implement preprocessing/training | Build IMC estimation framework | Build benchmark scripts       | All components underway |
| Sep 11 | First real dataset + MC runtime           | Train on real dataset            | Map NN architecture            | Benchmark MC + NN             | First real results      |
| Sep 12 | Validate MC across scenarios              | Accuracy evaluation/tuning       | Preliminary IMC estimates      | First comparison plots        | Initial comparison      |
| Sep 13 | Debug/integrate                           | Debug/integrate                  | Debug/integrate                | Combine results               | END-TO-END POC WORKS    |
| Sep 14 | Final/larger dataset                      | Final training                   | Final IMC analysis             | Systematic benchmarks         | Final experiments begin |
| Sep 15 | Runtime/validation experiments            | Accuracy/inference experiments   | IMC experiments                | Comparison plots              | Main results            |
| Sep 16 | Check financial validity                  | Finalize NN results              | Finalize assumptions/results   | Final tables/figures          | RESULTS FROZEN          |
| Sep 17 | MC explanation                            | ML explanation                   | IMC explanation                | Results explanation           | Build demo/script       |
| Sep 18 | Whole team                                | Record / edit                    | Review demo                    | Finalize figures/video        | VIDEO FINISHED          |
| Sep 19 | Final QA                                  | Final QA                         | Final QA                       | Final QA                      | SUBMIT                  |

## 9. Evaluation Plan

- Accuracy: MAE, RMSE, and  $R^2$  between neural-network predictions and Monte Carlo reference prices.
- Conventional performance: Monte Carlo runtime/throughput and NN CPU inference runtime/throughput.
- IMC analysis: estimated data movement, latency, energy, and/or throughput under documented assumptions.
- Scaling: if feasible, compare performance across different numbers of pricing queries.
- Accuracy by option region: if time permits, compare errors for OTM, ATM, and ITM scenarios.

## 10. Expected Final Result

The final presentation should answer three questions: (1) Can a small neural network approximate Monte Carlo option prices with acceptable error? (2) How much faster or more efficient is NN inference than repeatedly running Monte Carlo for the same class of pricing queries? (3) What additional benefit could IMC provide by reducing the cost of moving NN weights/data and performing matrix-vector operations?

The intended story is: Monte Carlo provides an accurate but computationally intensive reference workload; the neural network acts as a fast surrogate; IMC targets the neural network's remaining matrix-vector and data-movement costs.

## 11. Scope Rules / Minimum Viable Project

- European call options only for the minimum viable implementation.
- Synthetic option scenarios, not historical stock prediction.
- One small neural network before experimenting with larger architectures.
- Software/simulation-based IMC proof of concept; no physical chip is required.
- Do not add new features after Sep 16 unless a critical issue requires it.
- A real stock example is optional for the demo and should not become the main dataset.

## 12. What Everyone Needs to Do Today (Sep 9)

| Owner      | Today's deliverable                                                                                                                                |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Whole team | Agree on dataset columns, initial parameter ranges, NN architecture, shared repository/folder, and results format.                                 |
| Nini       | Get one Monte Carlo European-call example working; validate $S=100$ , $K=100$ , $T=1$ , $\sigma=.20$ , $r=.05$ ; turn it into a reusable function. |
| Rupsaa     | Build the NN loading/preprocessing/training skeleton using a dummy CSV with columns $S, K, T, \sigma, r, mc\_price$ .                              |
| Raya       | Select the IMC modeling methodology and identify exactly what metrics/assumptions are needed from the final NN.                                    |
| Ishika     | Create the common benchmark/result schema and starter functions/plots so results can be inserted later.                                            |

## 13. Shared File Formats

Dataset CSV:

`S,K,T,sigma,r,mc_price,moneyness,num_paths,mc_runtime_ms`

Common result table:

`method,mae,rmse,r2,latency_ms,throughput,data_movement_bytes,energy_estimate`

## 14. Internal Hard Deadlines

**Sep 13 — Complete end-to-end pipeline must work.**

**Sep 16 — Results frozen; focus shifts to communication and demo.**

Sep 18 — Demo video completely finished.

Sep 19 — Final quality check and submission.